Tema 6

Diseño y realización de pruebas

Entornos de Desarrollo

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Pruebas en el proceso de desarrollo de software 5
2.1 Planificación de pruebas a lo largo del ciclo de desarrollo 7
2.2 Tipos de pruebas: funcionales, estructurales, regresión, caja negra 7
2.3 Procedimientos y casos de prueba 10
3. Pruebas de código 11
3.1 Cubrimiento, valores límite, clases de equivalencia 11
3.2 Pruebas unitarias de clases y funciones 12
3.3 Uso de herramientas integradas en los entornos de desarrollo para realizar pruebas unitarias 13
3.4 Automatización de pruebas unitarias 15
3.5 Pruebas de integración 15
3.6 Diseño y documentación casos de prueba 15
4. Depuración de programas 17
4.1 Herramientas de depuración integradas en los entornos de desarrollo 17
4.2 Puntos de ruptura y seguimiento en tiempo de ejecución 18
4.3 Examinadores de variables 18
5. Resumen 20
Glosario 21
Bibliografía 22

Objetivos de la unidad

- Conocer la importancia de las pruebas de *software*.
- Aprender a diferenciar los distintos tipos de pruebas.
- Conocer el concepto de prueba unitaria, así como las herramientas proporcionadas por los entornos de desarrollo para realizarlas.
- Entender el proceso de depuración y el uso de los puntos de ruptura, así como aprender a consultar el valor de las variables durante la ejecución de los programas.

Tech Hub <FP> euroformac

1. Introducción

El **desarrollo de *software*** es un proceso complejo tanto desde el punto de vista técnico como del organizativo. Un proyecto está compuesto de varias fases que se llevan a cabo en un periodo de tiempo que puede ser extenso y en él suelen participar un gran número de personas. A todo esto, se le une el hecho de que el **diseño y la programación de *software*** son tareas que pueden tener un elevado nivel de dificultad.

Se puede deducir de lo expuesto anteriormente que el *software*, una vez construido, puede contener errores y fallos de todo tipo. Es indudable que esto es un problema, pero este se puede minimizar y una de las maneras de hacerlo es mediante la realización de pruebas.

Tech Hub <FP> euroformac

2. Pruebas en el proceso de desarrollo de *software*

En programación, a los errores se les conoce como "bugs" (bichos). Las pruebas contribuyen a eliminarlos.

Las pruebas de *software* existen desde que se escribió el primer programa informático. Es fácil imaginar que la primera vez que se resolvió un problema con un ordenador se comprobó, mediante una serie de pruebas, que los resultados obtenidos eran los esperados antes de considerar que el programa era fiable. Desde entonces, las pruebas de *software* han ido adquiriendo importancia y se han ido reforzando progresivamente con métodos y técnicas cada vez más avanzadas. Se puede afirmar que actualmente **las pruebas constituyen un conjunto de actividades fundamental en el desarrollo de *software*.**

Existen muchos tipos de pruebas. Cada uno persigue un objetivo distinto, como la constatación de que el *software* hace aquello para lo que fue diseñado (**pruebas funcionales**) o la comprobación de que su uso es intuitivo (**pruebas de usabilidad**). Hay muchas más, siendo las que buscan detectar errores las realizadas por los programadores.

Antes de comenzar a profundizar en el concepto y tipo de pruebas, es importante aclarar algunos aspectos relacionados con estas:

- **Los errores de *software* tienen coste**. Un sistema *software* que tenga errores va a generar una serie de costes. Estos tendrán que ver con el funcionamiento incorrecto (un importe de una factura mal calculado, por ejemplo), con el coste de la detección y solución del error y, en algunas ocasiones, con el coste reputacional de la empresa. **No hacer pruebas de *software* es costoso.**
- **Evitar los errores de software tiene coste.** Para evitar un error hay que tomar medidas preventivas, tanto para que no ocurran como para que sean detectados lo antes posible. Estas acciones suponen una dedicación de recursos humanos y técnicos que llevan asociado un coste. **Las pruebas de *software* son costosas.**

- **No todos los errores son iguales**. Es evidente que los errores no son iguales desde un punto de vista conceptual: no es lo mismo un fallo de cálculo que un error de ejecución. No obstante, resulta más interesante poner el foco en las consecuencias de los errores para establecer sus diferencias: es absolutamente distinto un error en el sistema de control de una central nuclear que en una máquina expendedora de refrescos, siendo fácil encontrar ejemplos que pongan de manifiesto la diferencia.

- **No existe el software sin errores**. Esta afirmación es una verdad a medias. En realidad, el *software* puede no contener errores, pero dependiendo de la complejidad y del tamaño del sistema, evitarlo puede ser tan costoso en tiempo y recursos que no sea posible. El esfuerzo dedicado a evitar los errores tiene que ver en muchas ocasiones con las consecuencias de estos. Es fácil intuir el esfuerzo que se realiza en las misiones espaciales para evitar errores de *software* dado el elevado coste que suponen sus consecuencias.

Establecer un correcto equilibrio a la hora de realizar pruebas de *software* o asumir que puede haber errores es complejo y dependerá de muchos factores, siendo uno predominante por encima de todos aquel que afecte a la salud o la integridad de las personas.

Las pruebas de *software*, como se puede ver a lo largo de esta unidad, cubren muchos aspectos y no todos tienen que ver con la detección de errores, pero esta detección es uno de sus principales objetivos.

Más info

Se puede obtener información histórica relacionada con los errores del *software* en el siguiente interesante artículo publicado en el periódico El País y escrito por un *Catedrático de la Universitat Politècnica de Catalunya.* Ver enlace.

2.1 Planificación de pruebas a lo largo del ciclo de desarrollo

En el limitado **ciclo de vida en cascada**, la "fase de pruebas" o de verificación se desarrollaba inmediatamente después de la "fase de implementación" y justo antes de la "fase de mantenimiento". Esta simplificación en el tratamiento de las pruebas implica suponer que todos los errores que se producen en un proyecto son de codificación y que se pueden resolver sin que otras fases del desarrollo se vean implicadas. Este planteamiento es erróneo.

En cambio, en el **ciclo de vida en V** cada fase de construcción del proyecto lleva asociada una fase de pruebas o de verificación, con el objetivo de no arrastrar errores entre fases y detectarlos en la fase más temprana posible. Este es el enfoque correcto en la realización de las pruebas.

Las pruebas de *software* no son una fase puntual que se realiza en un único momento del desarrollo de proyecto. La realización de pruebas es una tarea **transversal**, que se va a realizar a lo largo de todo el ciclo de vida del proyecto, desde su concepción hasta el mantenimiento.

2.2 Tipos de pruebas: funcionales, estructurales, regresión, caja negra

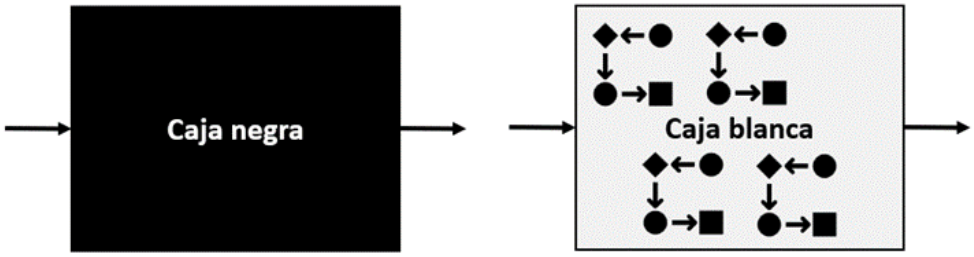
Existen muchos tipos de pruebas y diversas clasificaciones. En este apartado son presentadas algunas de ellas.

Una de las primeras clasificaciones distingue las pruebas en función de qué **perspectiva** se tiene del componente que se está probando. Hay dos tipos:

- **De caja negra.** Las pruebas pertenecientes a este tipo verifican que el componente que se está probando (una función, una clase, un módulo del sistema, etc.) funciona correctamente verificando que determinadas entradas generan las salidas correctas. Se dice que es de caja negra porque no se mira el interior del componente, sino

únicamente su funcionamiento desde el exterior. Por ejemplo, una prueba de caja negra de una lavadora consistiría en llenar el tambor de ropa sucia, poner detergente y suavizante, efectuar un lavado y comprobar que la ropa está limpia al finalizar.

- **De caja blanca.** El objetivo de las pruebas de este tipo es comprobar que, internamente, el componente está construido de manera correcta. En las pruebas de caja blanca se observa el interior del componente (también se les conoce como "**de caja de cristal**"), buscándose problemas de eficiencia o de seguridad, por ejemplo. Este tipo de pruebas puede realizarse con componentes que funcionan correctamente desde el punto de vista funcional con el objetivo de mejorarlos. También son conocidas como pruebas estructurales.



Las pruebas de caja negra y caja blanca pueden mostrar distintas perspectivas de un mismo componente

Otra posible gran clasificación tendría en cuenta el **ámbito** de las pruebas, dividiéndolas en:

- **Funcionales**. Verifican que el *software* hace aquello para que lo fue diseñado de manera correcta. Por ejemplo, en un sistema de administración de clientes de una empresa de telefonía, una posible prueba funcional consistiría en verificar que la contratación de una línea móvil a través de la web se realiza correctamente.
- **Técnicas**. Tratan aspectos técnicos que tendrán impacto en la eficiencia en el uso de recursos o en las capacidades del sistema. Por ejemplo, una prueba técnica podría consistir en verificar que una aplicación web es capaz de atender a un determinado número de usuarios concurrentes sin producir errores o que el uso de la memoria de un determinado proceso entra dentro de los límites establecidos.

Una última clasificación **divide las pruebas en función de cómo se llevan a cabo:**

- **Manuales.** Son realizadas y evaluadas de forma manual por personas.

- **Automatizadas**. Son realizadas y evaluadas de manera automática. Algunas pruebas de las consideradas automáticas han debido ser diseñadas y construidas previamente por personas, pero la ejecución de la propia prueba se puede llevar a cabo sin la participación humana.

Estas tres clasificaciones son un buen comienzo para ubicar cada uno de los múltiples tipos de pruebas en un lugar de partida.

En el siguiente listado se exponen algunos tipos de prueba junto con una breve descripción de su objetivo:

- **Pruebas unitarias.** De caja negra, de caja blanca y técnicas. Principalmente automatizadas. Son las realizadas por los programadores. Verifican que un elemento mínimo funcional de programa (una función, un método o una clase) funciona correctamente.
- **Pruebas de aceptación.** De caja negra, funcionales y, en ocasiones, técnicas. Manuales. Las suele realizar el equipo del cliente que ha contratado el desarrollo. Verifican que el sistema cumple con lo acordado. Estas pruebas sirven para que el cliente **acepte** el sistema como conforme a las especificaciones.
- **Pruebas de humo.** De caja negra y funcionales. Manuales y automatizadas. Son unas pruebas no exhaustivas que pueden ser realizadas por cualquiera de los actores que participan en el desarrollo (desde el cliente hasta los programadores). Buscan verificar que la funcionalidad principal del sistema funciona correctamente, pero sin entrar en una búsqueda minuciosa de fallos.
- **Pruebas de regresión.** De caja negra y funcionales. Principalmente automatizadas. Estas pruebas se realizan después de realizar cambios en el sistema (ya sean mejoras o corrección de errores), para comprobar que otras partes del sistema no se han visto afectadas.
- **Pruebas de rendimiento.** De caja negra y técnicas. Automatizadas. Este tipo de pruebas tiene como objetivo desde comprobar el funcionamiento del sistema respecto a diferentes cargas de trabajo (**pruebas de carga**) hasta someter al sistema a un volumen excesivo de trabajo para comprobar de qué manera se degrada su rendimiento (**pruebas de estrés**).

Más info

Se puede profundizar más en el conocimiento de las pruebas de rendimiento en el siguiente artículo: Ver enlace.

2.3 Procedimientos y casos de prueba

En un primer momento, la **realización de pruebas** puede parecer una terea sencilla, pero es una percepción equivocada. Las pruebas que se deben realizar en un sistema deben ser diseñadas y pensadas de forma exhaustiva, tratando de cubrir toda la casuística posible e intentando no dejar sin probar ningún componente que pueda posteriormente generar un problema.

La realización de pruebas es una disciplina conocida como **QA** (de ***Quality Assurance*** o **Aseguramiento de la Calidad**) o, coloquialmente, como *testing*. Los profesionales del *testing* son altamente cualificados y su labor tiene un importante reconocimiento en las empresas de desarrollo de *software*.

Una de las capacidades que debe tener un profesional que realiza pruebas tiene que ver con la definición de los **casos de prueba.**

Un **caso de prueba** es la especificación formal de un contexto de prueba de una unidad funcional concreta del sistema. En otras palabras, es la relación de pasos y acciones concretas que se deben ejecutar para realizar una prueba y verificar su resultado.

El objetivo final de los casos de prueba es disponer de un registro objetivo tanto de las pruebas como de los resultados obtenidos, para que ni los unos ni los otros quedan abiertos a la percepción de las personas que realizan las pruebas, ya que esto iría en contra de tener unos resultados fiables.

Más adelante, en esta misma unidad, se hace una presentación de la estructura de los casos de prueba y de un ejemplo de su uso.

3. Pruebas de código

Entre las muchas pruebas que se pueden realizar a un sistema *software*, algunas de ellas tienen como objetivo verificar que el código es correcto. Son las pruebas más próximas al programador, tanto que incluso es este perfil profesional el encargado habitualmente de hacerlas.

Las pruebas de código se pueden dividir en dos grandes grupos: **análisis estático y análisis dinámico.**

El **análisis estático** se realiza sobre el código sin ejecutar, buscando ineficiencias, errores de diseño, de seguridad o código muerto. En este tipo de pruebas se analiza el código desde un punto de vista estructural.

Por su parte el **análisis dinámico** se centra en comprobar que el código funciona correctamente, realizando las ejecuciones de diversos escenarios para detectar fallos en el funcionamiento. Las **pruebas unitarias** que se tratan en esta unidad pertenecen a este grupo de pruebas.

3.1 Cubrimiento, valores límite, clases de equivalencia

Hay una serie de conceptos que se utilizan en relación con las pruebas de código y que es importante conocer. Algunos de los más importantes son los siguientes:

- **Cubrimiento o cobertura.** Indica el porcentaje del código fuente que ha sido probado, disponiendo así de una métrica que determina el grado

de calidad de las pruebas. La cobertura puede ser referente al total de funciones, de ramas de bifurcación o de líneas de código.

- **Valores límites**. En las pruebas que se basan en los valores límite se utilizan como datos de entrada valores que se encuentran en los umbrales (los límites) aceptados por el componente que se está probando, asegurando así que el componente funciona en los valores extremos además de en los intermedios.
- **Clases de equivalencia.** Es una técnica consistente en agrupar todos los posibles parámetros de entrada de las pruebas en conjuntos homogéneos desde el punto de vista del componente que se va a probar. A estos conjuntos se les denomina **clase de equivalencia** y cualquier elemento dentro del mismo es representativo del resto del conjunto, por lo que la prueba que se realice con uno de los valores será válida para el resto de los elementos de la clase.

3.2 Pruebas unitarias de clases y funciones

Las **pruebas unitarias** son las que se realizan sobre pequeños bloques de código para verificar que su funcionamiento es correcto. Este tipo de pruebas las suelen diseñar, programar y realizar los programadores siendo una de las diversas habilidades que se les solicita como profesionales.

Las pruebas unitarias también se conocen como **test unitarios**: fragmentos de programa que preparan el entorno de ejecución, ejecutan el bloque de código que se desea probar y comprueba que el resultado obtenido es el correcto.

Una prueba unitaria puede probar indistintamente un método de una clase o una función, ya que conceptualmente ambos escenarios son similares.

Existen librerías que facilitan la creación de las pruebas unitarias. En Java, por ejemplo, es muy popular y utilizado el *framework* de pruebas ***Junit***. En Python, ***unittest*** viene incluido como una librería estándar del lenguaje, por lo que no requiere instalación adicional.

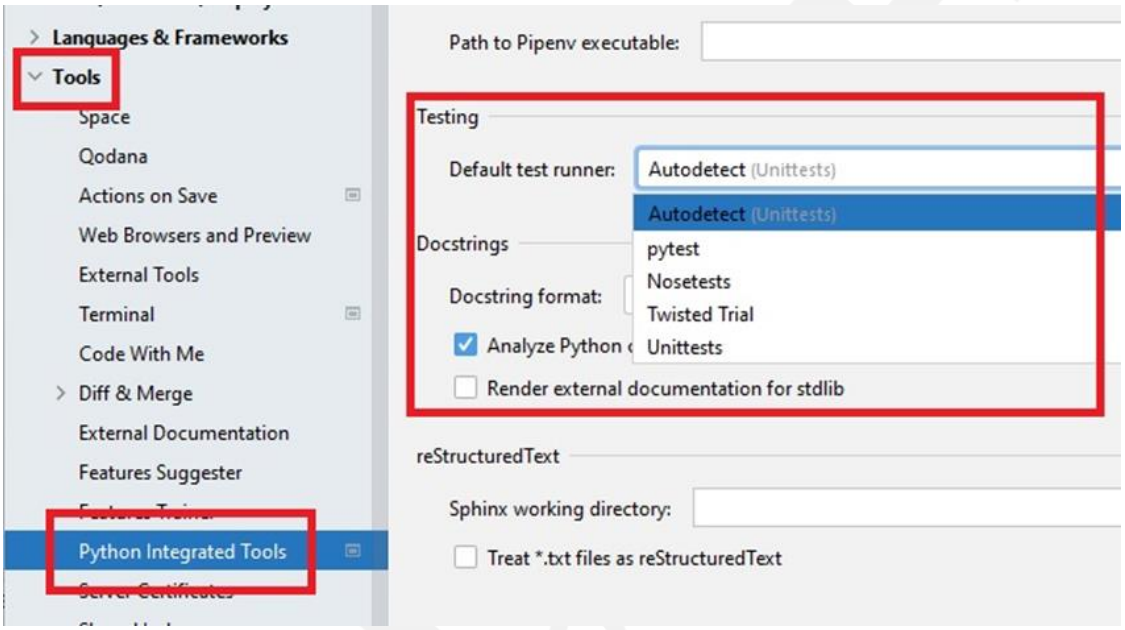
Más info

Se puede profundizar más en el conocimiento de las pruebas unitarias en Python en el siguiente enlace: Ver enlace.

3.3 Uso de herramientas integradas en los entornos de desarrollo para realizar pruebas unitarias

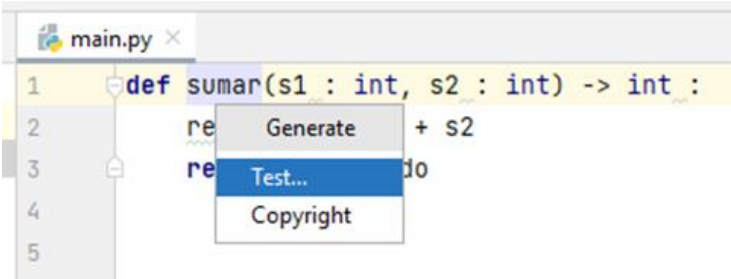
Los **entornos de desarrollo integrados (IDE)** suelen incorporar herramientas o accesos directos a herramientas de realización de pruebas unitarias.

En el caso de **PyCharm**, el popular IDE de desarrollo para Python, la selección de la herramienta de test se realiza a través de la ventana accesible desde la opción *Settings* del menú *File*. Por defecto aparece seleccionada la opción *Autodetect(Unittests)*.



Selección del entorno de pruebas de PyCharm

Como se puede esperar, la construcción de las pruebas también está soportada por los IDE. En el caso de PyCharm, pulsando, por ejemplo, sobre el código fuente con el botón derecho y seleccionado *Generate* aparece una opción para la creación de pruebas, tal y como se puede observar en la siguiente captura.



Generación de pruebas unitarias

Este proceso solicitará que se seleccione sobre qué elementos (funciones o métodos) se desea crear pruebas y construirá una clase con tantos métodos como pruebas a realizar. Será labor del desarrollador de las pruebas el rellenar dichos métodos.

```
main.py   test_main.py
1  from unittest import TestCase
2
3  class Test(TestCase):
4      def test_sumar(self):
5          self.fail()
6
```

Clase de pruebas autogenerada

En la siguiente figura se muestra la prueba unitaria anterior modificada para que realice el test deseado.

```
main.py   test_main.py
1  from unittest import TestCase
2  from main import sumar
3
4  class Test(TestCase):
5      def test_sumar(self):
6          resultado_esperado = 13
7          resultado_obtenido = sumar(5,8)
8          self.assertEqual(resultado_obtenido, resultado_esperado)
```

Ejemplo de prueba unitaria programada

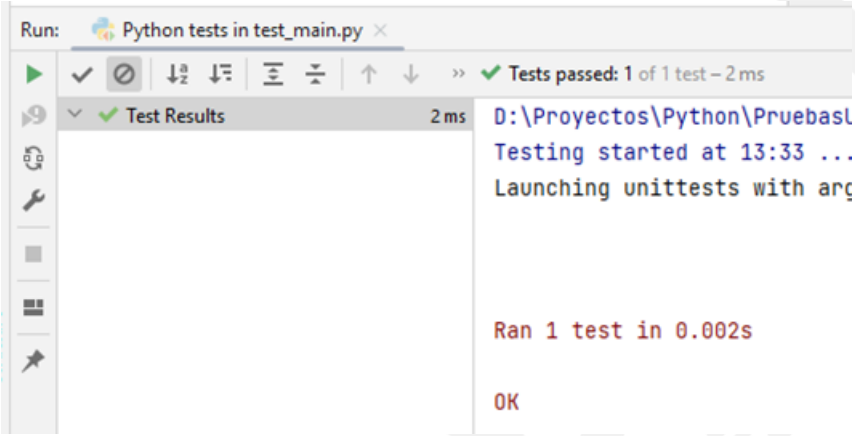
Como se puede observar, el método ***test_sumar*** hace uso de la función ***sumar*** que es el código que se quiere probar. Se ejecuta una operación y se comprueba que el resultado obtenido coincide con el esperado mediante el método ***assertEqual***.

Más info

Se puede aprender más sobre la creación de test unitarios en Python en el siguiente artículo: Ver enlace.

3.4 Automatización de pruebas unitarias

Una vez programada la prueba, tal y como se pudo observar en la sección anterior, comienza el proceso de ejecución de esta. En PyCharm basta con ejecutar la clase o clases que tienen los tests. El editor detecta que se trata de una prueba y muestra el resultado de esta, además de más información relacionada con el contexto de las pruebas.



Resultados de la ejecución de una prueba unitaria con éxito

3.5 Pruebas de integración

Las **pruebas unitarias** proporcionan certezas sobre el funcionamiento del código en entornos acotados y muy limitados. Estas pruebas son imprescindibles, ya que son el primer paso para poder asegurar que el código desarrollado está libre de errores y es funcional.

Lamentablemente, no es suficiente. Los programas casi siempre funcionan dentro de sistemas más complejos. Se relacionan con otros sistemas o módulos, de la misma empresa o de terceros. Es necesario comprobar que los programas funcionan bien en el entorno en el que realmente se van a ejecutar. Las **pruebas de integración** permiten verificar que el componente de software que se está probando funciona correctamente en lo referente a las relaciones que va a tener con otros componentes, módulos, subsistemas o sistemas.

3.6 Diseño y documentación casos de prueba

Un caso de prueba está compuesto por los siguientes elementos:

- **Identificador.** Un identificador único que permita referenciar al caso de prueba de manera inequívoca.

- **Descripción.** Descripción textual de qué persigue verificar la prueba y sobre qué componente.
- **Precondiciones.** Reflejo el estado en el que debe estar el sistema antes de realizar la prueba.
- **Datos de prueba.** Datos que se utilizarán para realizar la prueba.
- **Pasos.** Secuencia de acciones a realizar para ejecutar la prueba.
- **Resultado esperado.** Especificación precisa del resultado que debería producirse tras la ejecución de la prueba para comprobar que esta se ha realizado satisfactoriamente.
- **Resultado obtenido**. Una vez ejecutada la prueba, refleja el resultado obtenido de la ejecución de los pasos.
- **Estado.** Resultado de la prueba, como consecuencia de enfrentar los resultados esperados y los obtenidos. El estado puede ser **CORRECTO** o **ERROR**.

La siguiente figura representa un posible diseño de caso de prueba, aunque tanto el formato como los elementos que lo forman pueden cambiar, ya que lo importante es que las pruebas se ajusten a las necesidades del sistema.

IDE	Descripción	Precondiciones	Datos de prueba	Pasos	Resultado esperado	Resultado obtenido	Estado
01	Registro de usuario en el sistema	El usuario no existe en la base de datos	Identificador: campo@gmail.com Contraseña: p@t3_Bc	Se introducen las credenciales en el formulario y se pulsa el botón "registrar".	El usuario queda registrado en la base de datos.	Se ha mostrado una ventana con el mensaje de error "Usuario desconocido"	ERROR
02	Eliminación de usuario	El usuario debe existir en la base de datos	Un identificador de usuario existente	Se introduce el identificador del usuario y se pulsa el botón "eliminar"	El usuario queda eliminado de la base de datos y se muestra un mensaje informando.	El usuario queda eliminado de la base de datos y se muestra un mensaje informando.	CORRECTO

Ejemplo de descripción de casos de prueba

Como se puede observar en el ejemplo anterior, los casos de pruebas proporcionan información precisa de cómo realizar las pruebas y cómo constatar que estas han sido superadas con éxito o por el contrario no ha sido así. De esta manera, queda constancia escrita de qué se probó, cómo y el resultado obtenido.

4. Depuración de programas

Construir un programa es una tarea que puede resultar difícil. Se utiliza un **lenguaje formal** con una **sintaxis muy concreta**, hay que conocer bien el problema que se quiere resolver y es necesario tener una alta capacidad de abstracción para 'imaginar' cómo debe ser el programa para poder escribirlo.

Si a pesar de todos estos inconvenientes se ha conseguido programarlo libre de errores, pero no funciona correctamente ¿qué hacer? Una de las opciones consiste en realizar un proceso conocido como **depuración**, que consiste en ejecutar el programa de tal manera que se pueda "observar" la ejecución.

La depuración permite ejecutar un programa línea a línea, teniendo acceso a la memoria para saber con precisión que ocurre en cada momento, con el objetivo de detectar errores, ineficiencias o, simplemente, comprender el código fuente.

4.1 Herramientas de depuración integradas en los entornos de desarrollo

Todo entorno de desarrollo mínimamente serio dispone de herramientas de depuración. Ya sean externas integradas o propias del entorno, los IDE proporcionan tanto el entorno como las utilidades necesarias para hacer depuración o ***debug***, como también se conoce habitualmente a este proceso.

4.2 Puntos de ruptura y seguimiento en tiempo de ejecución

Un **punto de ruptura** o ***breakpoint*** es una indicación que se adjunta a una línea de código para notificar al depurador que al llegar a dicha línea debe suspender la ejecución.

Un programa puede tener todos los puntos de ruptura que se consideren necesarios y estos pueden incluir condiciones que determinen si debe activarse o no.

En **Visual Studio Code**, cuando se utiliza para programar en Python, los puntos de ruptura se crean pulsando a la izquierda del número de línea. El punto rojo indica que en esa línea hay un punto de ruptura. Para eliminarlo, basta con pulsar sobre la propia marca.

```
3   if n1>n2:
4       mayor=n1
5   else:
6       mayor=n2
7   if n3>mayor:
```

Puntos de ruptura en Visual Studio Code

Para que el punto de ruptura tenga efecto hay que realizar la ejecución en modo "depuración". En Visual Studio Code esto se hace pulsando la opción "*Start Debugging*" o la tecla "F5". El depurador se detendrá en el primer punto de ruptura encontrado, pudiendo entonces el usuario avanzar línea a línea en la ejecución pulsando la tecla "F10".

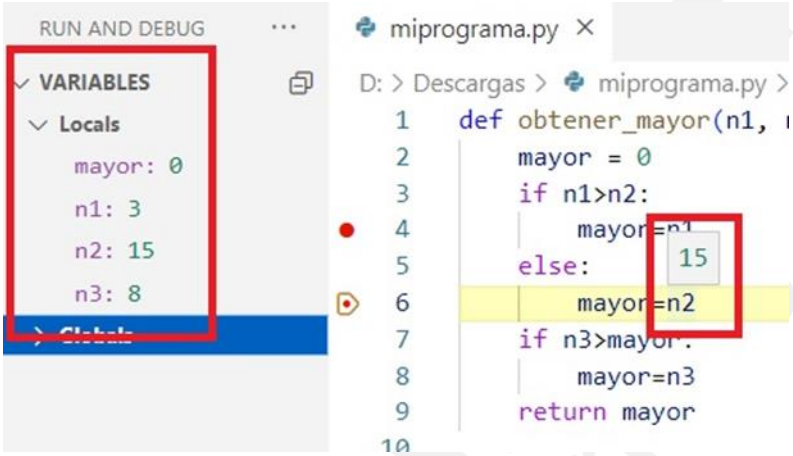
```
4       mayor=n1
5   else:
6       mayor=n2
7   if n3>mayor:
```

Ejecución detenida en un punto de ruptura

4.3 Examinadores de variables

Otra de las características importantes de los depuradores es el **inspector de variables**. Permite comprobar el valor de las variables cuando el programa se encuentra detenido en un punto de ruptura.

En la siguiente captura se pueden observar el valor de las variables de dos maneras posibles: una en el área izquierda, en una sección denominada "VARIABLES" en las que aparecen las variables locales y globales; la otra alternativa se consigue posicionando el cursor del ratón sobre el nombre de la variable en el propio código, apareciendo el valor en un mensaje flotante (en el ejemplo la variable **n2** tiene el valor **15** en el momento en el que el programa se detuve en el punto de ruptura).



Inspección de valores de variables

Más info

En el siguiente vídeo se puede aprender más sobre depuración de programas Python en Visual Studio Code: Ver vídeo.

5. Resumen

La **calidad del *software*** tiene que ser un objetivo que esté siempre en la mente de los participantes en su construcción. La detección y el diagnóstico de errores forma parte del proceso de aseguramiento de la calidad. Existen técnicas y herramientas para realizar pruebas, siendo las pruebas unitarias las realizadas por los programadores.

Una vez detectado un error hay que descubrir la causa. El **proceso de depuración** es una manera para estudiar en ejecución los programas, permitiendo conocer su funcionamiento interno y detallado. Los **entornos de desarrollo** integran depuradores que facilitan dicho estudio, permitiendo realizar la labor de detección de errores de manera sencilla.

Tech Hub <FP> euroformac

Glosario

- **Bug**. Nombre informal con el que se conocen a los errores de software
- **Código muerto.** Cualquier conjunto de líneas de código que nunca van a ejecutarse debido a las condiciones o estructura del programa.
- **Ineficiencia del *software*.** Se dice que un elemento de software es ineficiente cuando utiliza más recursos de los necesarios para resolver la problemática para la que está programado.
- **Ramas de bifurcación de un programa.** Cada uno de los posibles caminos que puede tomar un programa durante su ejecución en función de las condiciones contenidas en él.

Tech Hub <FP> euroformac

Bibliografía

País, E. E. (2023, April 6). EL PAÍS: el periódico global. El País. https://www.elpais.es/

Impulse la experiencia de sus usuarios - Atentus: Servicio de Monitoreo. (2022, June 9). Atentus: Servicio De Monitoreo. https://atentus.com/

Python, R. (n.d.). Python Tutorials – Real Python. https://www.realpython.com/

Inicio. (n.d.). El Libro De Python. https://www.ellibrodepython.com/

Tech Hub <FP> euroformac